



TECHNISCHE
UNIVERSITÄT
WIEN

ottobock.

Framework for System Simulation

Thesis – Design and Implementation

Florian Frech, 01.12.2025

1. Master Pendulum

- Parameter Synchronization
- Verification Cases

2. Structural Analysis

- FMU Model Structure
- System Graphs and Execution Order

3. Co-Simulation Orchestration Algorithms

- Motivation & Co-Simulation Basics
- Jacobi Co-Simulation Orchestration Algorithm
- Gauss-Seidel Co-Simulation Orchestration Algorithm

4. Interface Jacobian-based Co-Simulation (IJCSA)

- Theory
- Local Interface Jacobian-based Co-Simulation
- Global Interface Jacobian-based Co-Simulation
- Algebraic Loop Example

5. Controlled Pendulum Demos

- Simple Coupled System
- Complex Coupled System
- Monolithic Comparison Simple vs. Complex

6. Co-Simulation Algorithm Verification

- Overall Trajectory Behavior
- Error Convergence Behavior

7. Next Steps

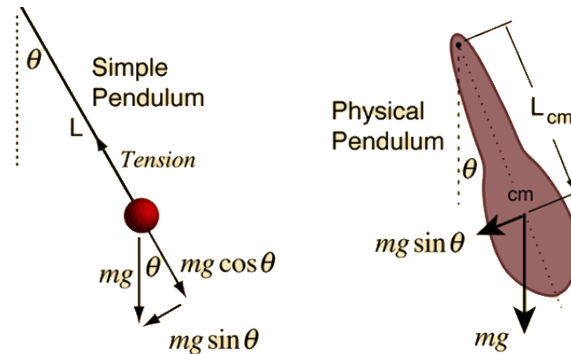
Pendulum Dynamics Governing Equation:

$$\alpha = \frac{d\omega}{dt} = \frac{d^2\theta}{dt^2}$$

$$\alpha = \frac{\tau}{I}$$

$$\alpha = \frac{\tau_{grav} + \tau_{drive}}{I}$$

$$\alpha = \frac{m \cdot g \cdot L_{cm} \cdot \sin(\theta) + \tau_{drive}}{I}$$



<http://hyperphysics.phy-astr.gsu.edu/hbase/pend.html>

Parameter Synchronization

- The mass distribution of the pendulum is defined by the 2D geometry with thickness t
- If the Modelica / FMU, OpenSim, and FEM pendulum should be equivalent rigid body models of the pendulum, the **point mass assumption for the inertia I of the pendulum fails**:

$$I_{FEM} \neq m_{FEM} \cdot L_{cm}^2$$

- Therefore, **the inertia I computed by the FEM pendulum is directly synchronized** to the equation-based and OpenSim pendulum implementations to **ensure consistent dynamics** across all pendulum models.
- For all three models to **produce equivalent accelerations** under the same **angular position θ applied torques τ** the following parameters are synchronized from the FEM pendulum to the equation-based and OpenSim pendulums:

$$\alpha = \frac{m \cdot g \cdot L_{cm} \cdot \sin(\theta) + \tau_{drive}}{I}$$

- Same total mass m
- Same distance L_{cm} from the pivot point to the center of mass
- Same moment of inertia I about the pivot point

FEM Pendulum

- Mass:** The FEM pendulum computes the total mass by integrating the mass density over the pendulum's 2D geometry and multiplying by the thickness.

$$m = \int_{\Omega_p} \rho \cdot t \, dA = \rho \cdot t \cdot A$$

- Center of Mass:**

$$c_x = \frac{\int_{\Omega_p} \rho \cdot t \cdot x \, d\Omega_p}{m}, \quad c_y = \frac{\int_{\Omega_p} \rho \cdot t \cdot y \, d\Omega_p}{m}$$

- Distance Center of Mass to pivot point:**

$$L_{cm} = \sqrt{c_x^2 + c_y^2}$$

- Moment of Inertia around pivot point:**

$$I = \int_{\Omega_p} \rho \cdot r^2 \, d\Omega_p \quad \text{with} \quad r = \sqrt{x^2 + y^2}$$

Modelica / FMU:

- Primary **incorrect version used assumption $I = m \cdot L_{cm}^2$**

```
equation
  der(q)      = omega;
  der(omega) = -(g / L_cm) * sin(q) + torque / (m * L_cm^2); // WRONG!
```

- Correct Modelica Implementation:**

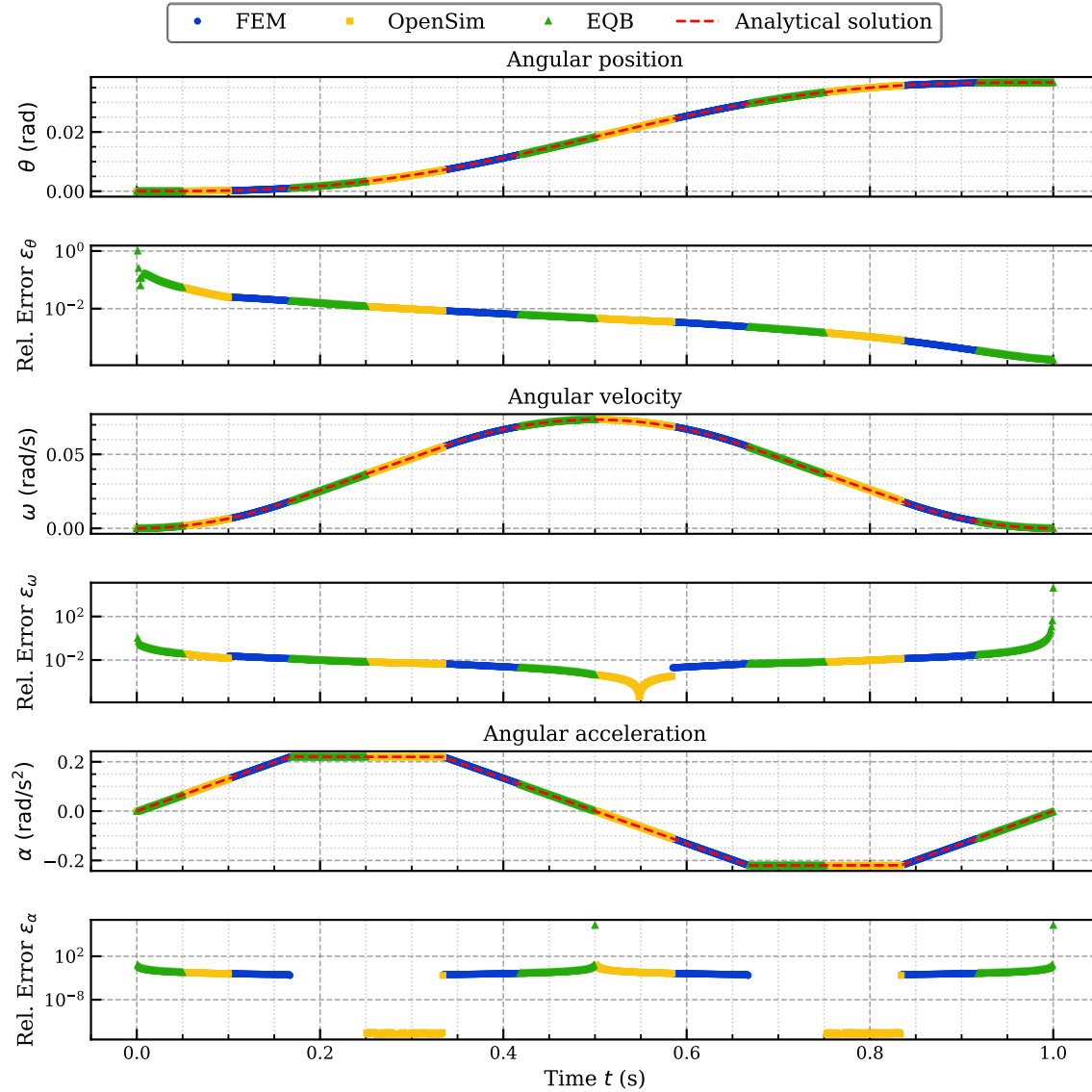
```
equation
  der(q)      = omega;
  der(omega) = -(m * g * L_cm / I) * sin(q) + torque / I;
```

OpenSim:

- Parameters to set: L_{cm} , m_{head} , I_{head}
- Pendulum modeled as sphere with mass m , connected by a massless rigid link L_{cm} to the pivot point
- Head with inertia I_{head} around its center of mass (**must be determined**)
- Total Inertia:** $I_{tot} = I_{FEM} = I_{head} + m \cdot L_{cm}^2$
- Head Inertia:** $I_{head} = I_{FEM} - m \cdot L_{cm}^2$

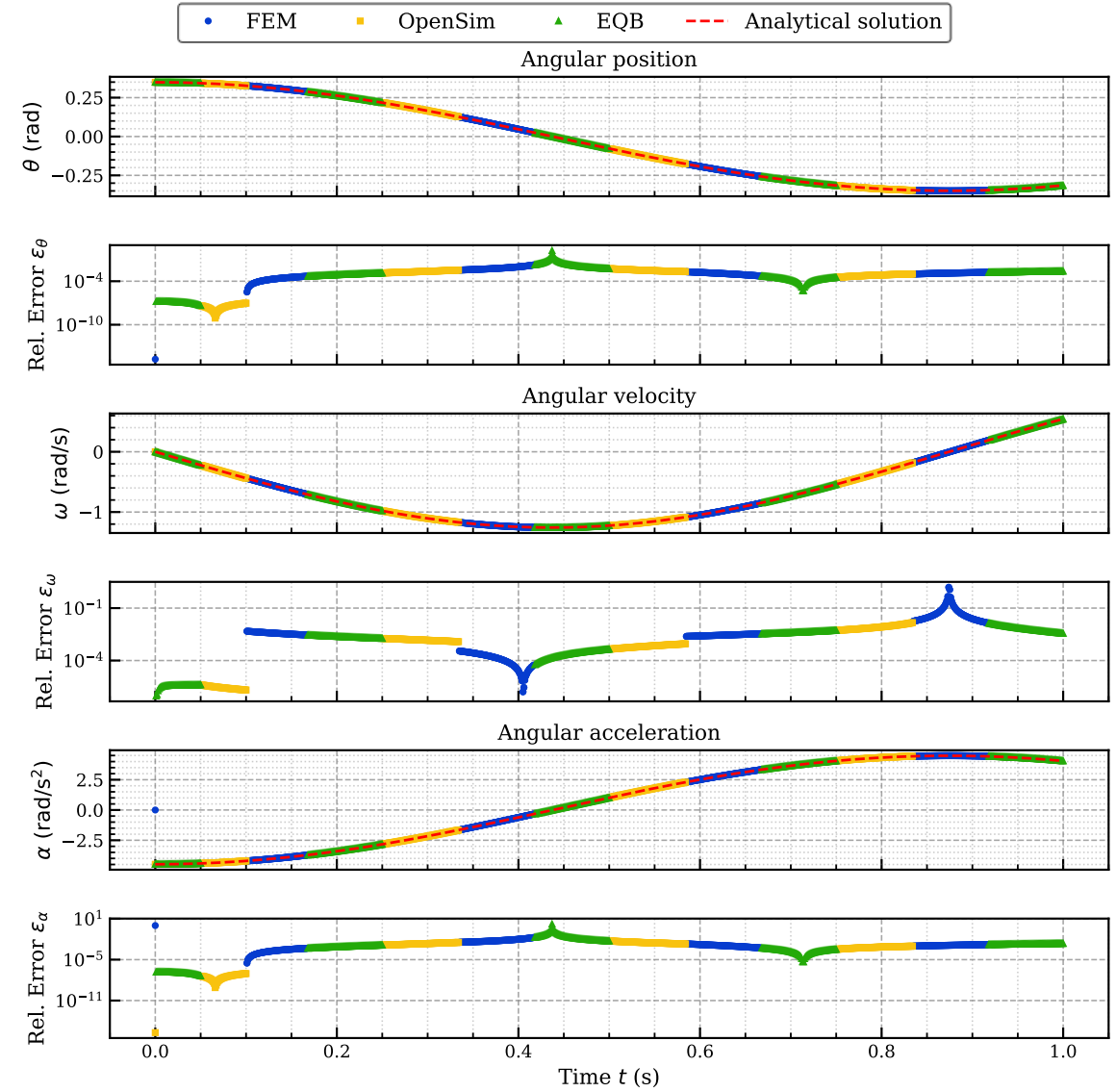
Pendulum: FEM vs OpenSim vs EQB

g : off, contact: off, torque: ramp, $\Delta t = 0.001$ s



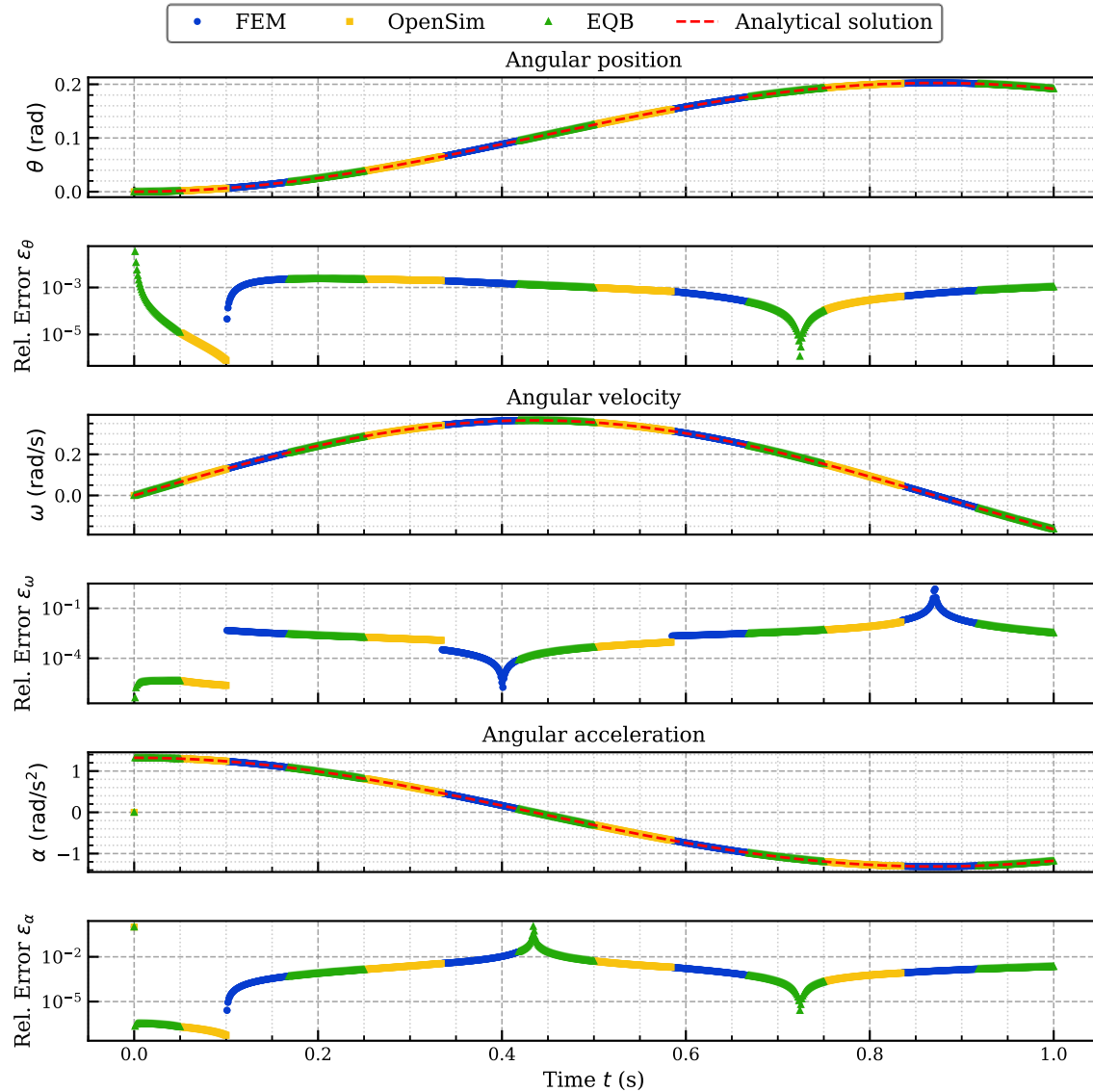
Pendulum: FEM vs OpenSim vs EQB

g : on, contact: off, torque: None, $\Delta t = 0.001$ s



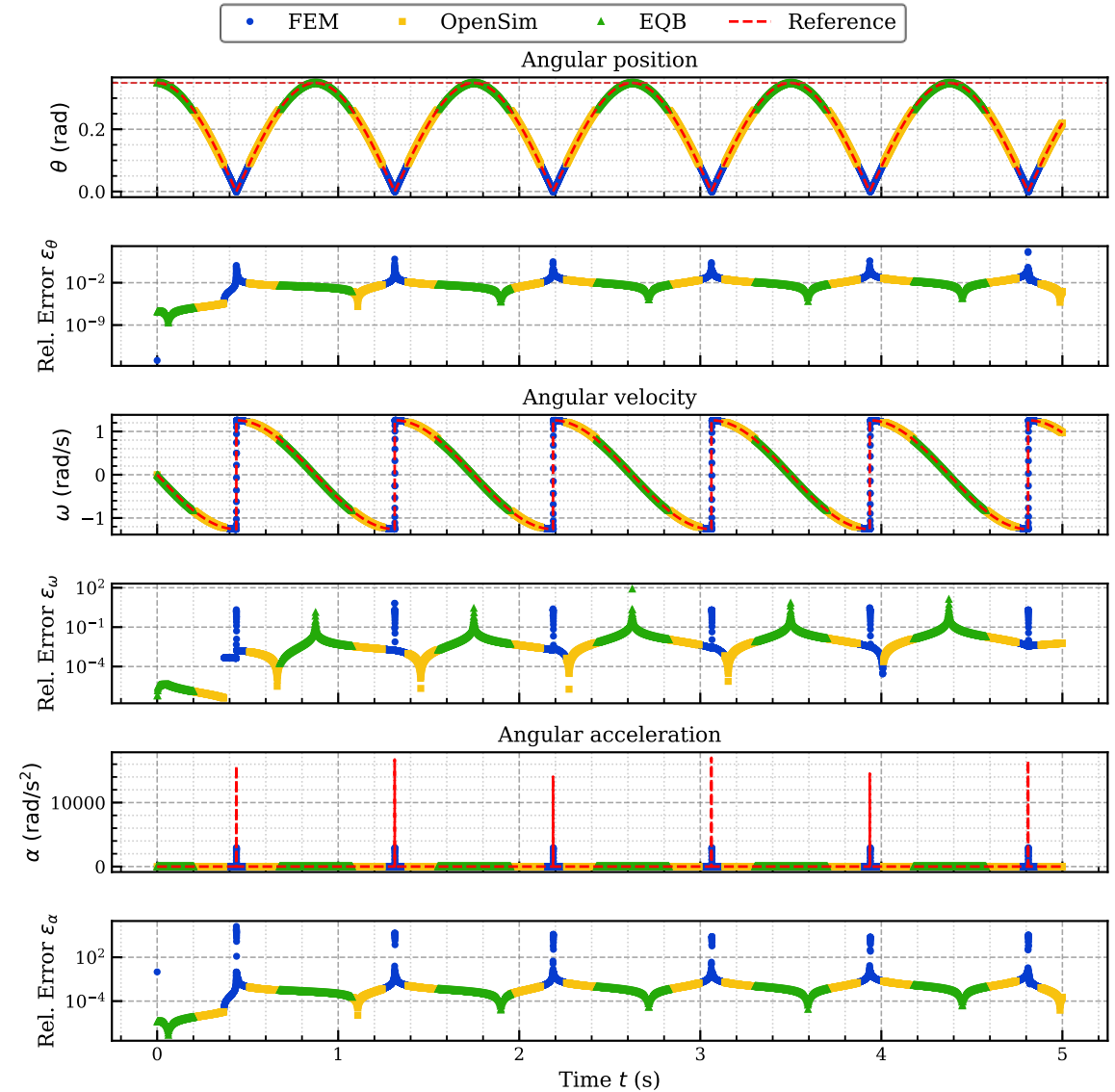
Pendulum: FEM vs OpenSim vs EQB

g : on, contact: off, torque: constant, $\Delta t = 0.001$ s



Pendulum: FEM vs OpenSim vs EQB

g : on, contact: on, torque: None, $\Delta t = 0.001$ s



FMU Model Structure:

- FMUs exported from OpenModelica are of version 2.0 and provide information about the **model variables and model structure**
- **Variable causality**: input, output, parameter, calculated parameter, local, etc.
- **Variable dependencies** in the model structure section:
 - which output depends on which variables
 - which derivative depends on which variables
 - which initial unknown depends on which variables

Model Structure for drive FMU:

```
<ModelStructure>
  <Outputs>
    <!-- Output: torque (index 30) -->
    <Unknown
      index="30" name="torque"
      dependencies="springDamper.phi_rel, springDamper.w_rel, alpha, omega, phi"
      causality="(state), (state), input, input, input"
    />
  </Outputs>
</ModelStructure>
```

Model Structure for pendulum FMU:

```
<ModelStructure>
  <Outputs>
    <!-- Output: alpha (index 6) - angular acceleration -->
    <Unknown
      index="6" name="alpha"
      dependencies="_D_outputAlias_q, torque"
      causality="(state), input"
    />
  </Outputs>
</ModelStructure>
```

System Graphs

- **self.graph**: full connection graph
 - An edge for every connection from **src_comp.scr_port** → **dest_comp.dest_port**
- **self._dag**: directed acyclic graph (reduced zero-delay containing only true direct-feedthrough dependencies)
 - An edge is only built for a connection where the **destination component has a direct-feedthrough output** that depends on the destination input port
 - Captures the **instantaneous dependencies** (no state-delay)

Execution Order and Generations

- Derive a **parallelizable execution schedule** that
 - respects all zero-delay dependencies
 - exposes independent components/groups that can be stepped in parallel
 - pushes purely state based outputs to the end.
- **Condensation of zero-delay graph** collapses each strongly connected component into a single node
- **Topological ordering** returns layers of SCCs in causal order (earlier generations depend on previous ones)
- Expanding SCCs into component generations: for each SCC in a generation, it collects its member components and appends them to the current generation list.
- Pure dynamic producers are summarized in a final generation which ensures that they use the most recent loop outputs (e.g., drive sees already new u_control for new state computation)

Algebraic Cycles and Strongly Connected Components (SCC)

- After building the directed acyclic loop, ***nx.strongly_connected_components(self._dag)*** is used
- Any SCC with more than one node, or a single node with a self-loop is considered an algebraic loop / strongly coupled set and is stored in a list of component names
- There exists a directed cycle of zero-delay edges (some outputs depend instantaneously on inputs that depend back on those outputs)

(Blochwitz, 2012)

Motivation:

- **Core Concept:** Simulate a global system by composing multiple independent subsystems, each potentially using different simulation tools or solvers
 - Computing the behavior of the combined models over time.
 - Each **simulation unit** with its own interface for **getting, setting inputs/outputs and computing the behavior** of its model over a given interval of time.
- **Master Algorithm:** scheduling **execution and communication** of each co-simulation unit
 - Representing a system assembled from models in various domains; each with their own appropriate simulator
 - Black-box-models which hide internal details
- **Problem:** difficult to ensure that the results produced by a co-simulation can be trusted
 - Communication frequency between different differential-equation-based units
 - Numerical properties of the participating units
 - Users do not always know how to configure the co-simulation

State Space Representation of Dynamical System:

$$\dot{x} = f(x, u, t), \quad y = g(x, u, t)$$

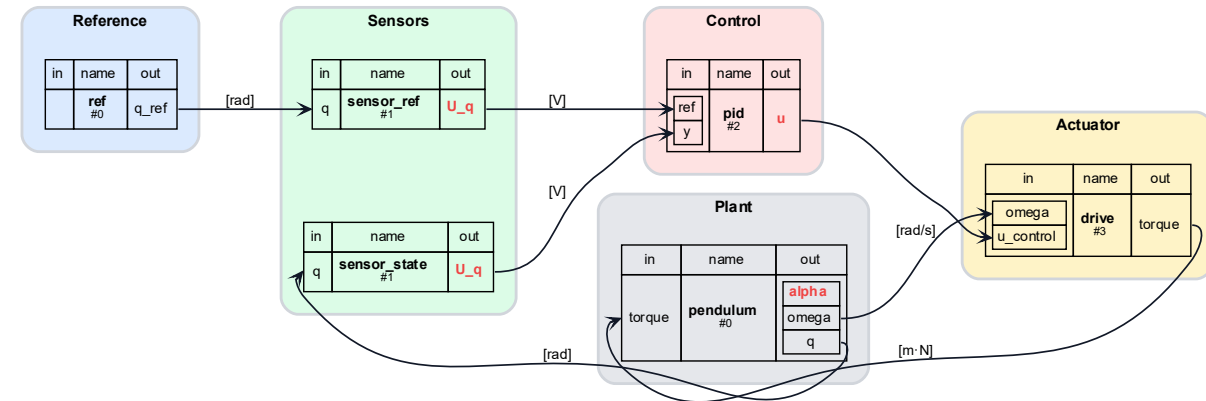
- $\dot{x}$: time derivative of the state
- f : function that describes the state evolution
- g : function that maps the state and inputs to the outputs
- **Behavior:** Set of trajectories $(x(t), y(t))$ that satisfy the state evolution equations under a given experimental frame (assumptions, boundary conditions, input signals, etc.)
- **Simulator/Solver:** Numerical algorithm that approximates trajectories $(x(t), y(t))$
 - Accuracy depends on numerical method, step size and step size control, representation and discretization of continuous signals (interpolation, sample-and-hold, etc.)
- **Simulation Unit (SU):** package of a dynamical system model and a numerical solver
- **Co-Simulation:** Simulation of coupled system composed from several SUs S_i
- **Orchestrator/Master** manages simulated time, schedules SUs, routes outputs to inputs according to a **co-simulation scenario** (connection graph)

Continuous-Time Simulation Units and Communication Grid

$$S_i = \langle X_i, U_i, Y_i, \delta_i, \lambda_i, x_i(0), \phi_{U_i} \rangle$$

- X_i, U_i, Y_i : are the state, input, and output spaces,
- δ_i : is the internal state transition over one macro step (including its numerical solver)
- λ_i : maps the state and inputs to outputs
- $x_i(0)$: initial state
- ϕ_{U_i} : is the interpolation/extrapolation scheme used to approximate $u_i(t)$ between communication points (e.g., zero-order hold, linear, higher order)
- Continuous-time (CT) SUs **internally integrate ODE/DAE systems**
- Each SU S_i may use its own internal step size h_i and its **own solver**
- Orchestrator defines **communication grid** with macro step size H with the communication times $t_n = t_0 + nH, \quad n = 0, 1, 2, \dots$ (**SUs exchange inputs/outputs**)
- Between t_n and t_{n+1} each **SU advances independently** using internal micro-steps h_i , while seeing its **input approximated over the macro step** (e.g., zero-order hold, linear, higher order)

Simple Coupled Pendulum



(Gomes, 2018; Broman, 2013))

Motivation:

- We consider one communication step from t_n to t_{n+1} with macro step size H
- Simulation Units (SUs) with internal state x_i , inputs u_i and outputs y_i
- The **coupling equations** $L(y, u)$ be a set of equations relating all SUs inputs and output to each other

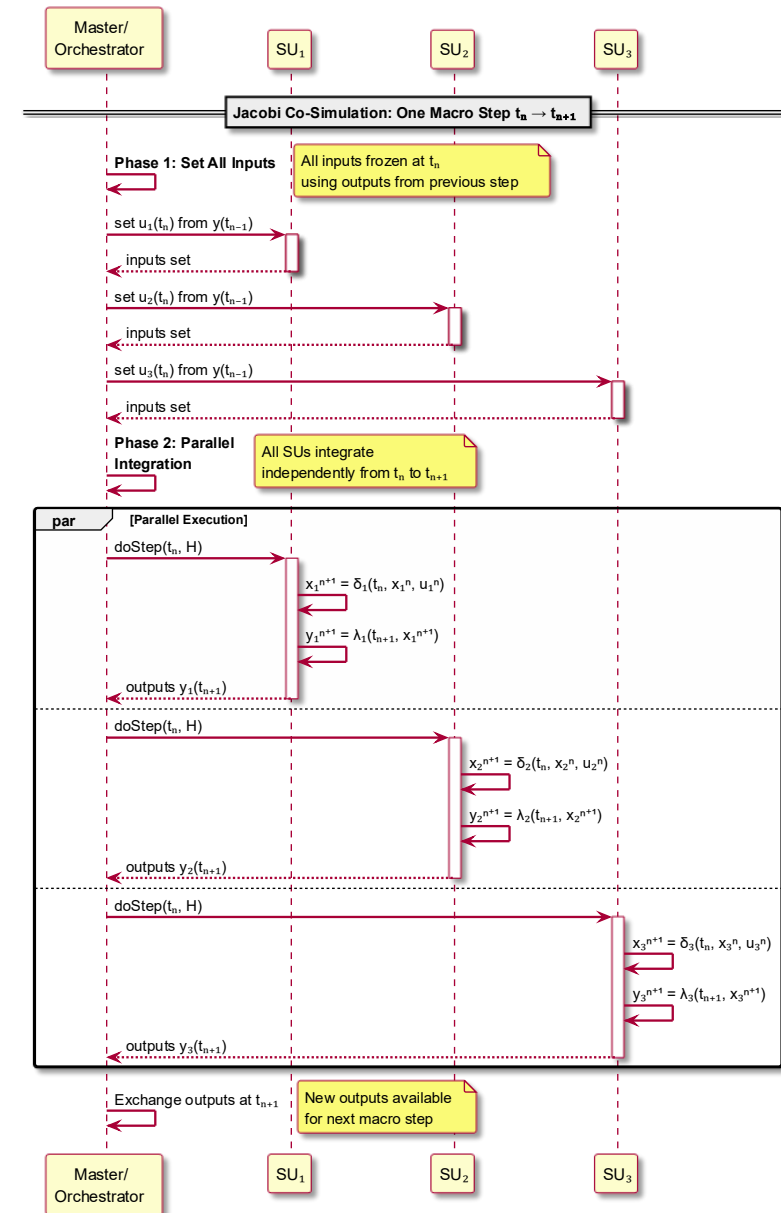
Jacobi-Co Simulation Orchestration Algorithm

- **Explicit, non-iterative** method where all **SUs are advanced in parallel** using input values from the previous communication point
1. At communication time t_n each SU has state x_i^n and input u_i^n . The inputs at the next step u_i^{n+1} are defined by the couplings using only outputs at time t_n (explicit coupling)
 2. For the next macro step of H :
 - Orchestrator algorithm freezes each SU's input as some extrapolated function (e.g., zero-order hold)
 - All SUs integrate in parallel from $t_n \rightarrow t_{n+1}$ using the frozen inputs, producing new states x_i^{n+1} and outputs y_i^{n+1}

$$x_i^{n+1} = \delta_i(t_n, x_i^n, u_i^n), \quad y_i^{n+1} = \lambda_i(t_{n+1}, x_i^{n+1}, u_i^{n+1})$$
 3. At t_{n+1} , the new outputs y_i^{n+1} are available and used to compute the inputs for the next step u_i^{n+2} via the coupling equations

Properties

- **Parallelism:**
 - All SUs can run concurrently between t_n and t_{n+1}
 - All inputs frozen at time t
- **Simplicity:**
 - Execution order NOT required
 - Explicit one-step method
 - No interface iterations or rollback; suitable for loosely coupled systems.
- **Phase lag / extrapolation error:**
 - Each SU sees "old" information about others; in closed loops this introduces effective delays and can degrade accuracy or stability
 - All components advance independently



(Broman, 2013))

Motivation:

- We consider one communication step from t_n to t_{n+1} with macro step size H
- Simulation Units (SUs) with internal state x_i , inputs u_i and outputs y_i
- The **coupling equations** $L(y, u)$ be a set of equations relating all SUs inputs and output to each other

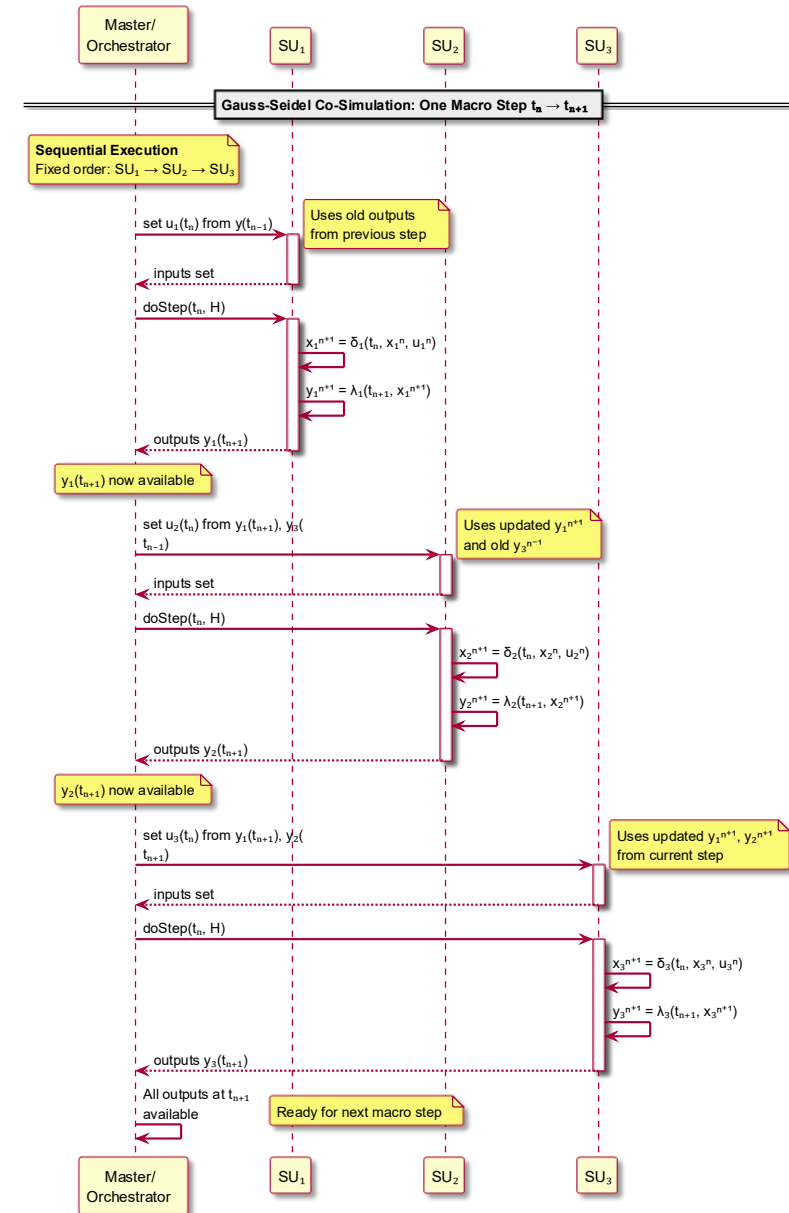
Gauss-Seidel Simulation Orchestration Algorithm

- The Gauss-Seidel scheme is sequential and uses the most recent information inside a macro step
1. Fix an order of SUs, e.g., $S_1, S_2, \dots, S_N$
 2. At communication time t_n :
 - S_1 is advanced first from $t_n \rightarrow t_{n+1}$ with inputs extrapolated from data at t_n or earlier

$$x_1^{n+1}, y_1^{n+1} = \text{step}(S_1, x_1^n, u_1^n(\cdot))$$
 - S_2 is advanced next. Its inputs may now use y_1^{n+1} , which has just been updated by S_1
 3. After all SUs have been advanced, time is increased to t_{n+1}

Properties:

- **Early SUs** in the fixed order **see old outputs** from SUs that come later in the order
- **Late SUs** in the fixed order **see new outputs** from SUs that are earlier in the order
- **Reduced phase lag compared to Jacobi**: some couplings use updated outputs within the same macro step.
- Potentially **better stability** for certain physical orderings (e.g., integrating light masses before heavy masses, or input–output chains in causality order).
- **Loss of parallelism**: SUs must be advanced sequentially in the chosen order.
- **Order sensitivity**: The chosen sequence can significantly affect accuracy and stability.



(Broman, 2013))

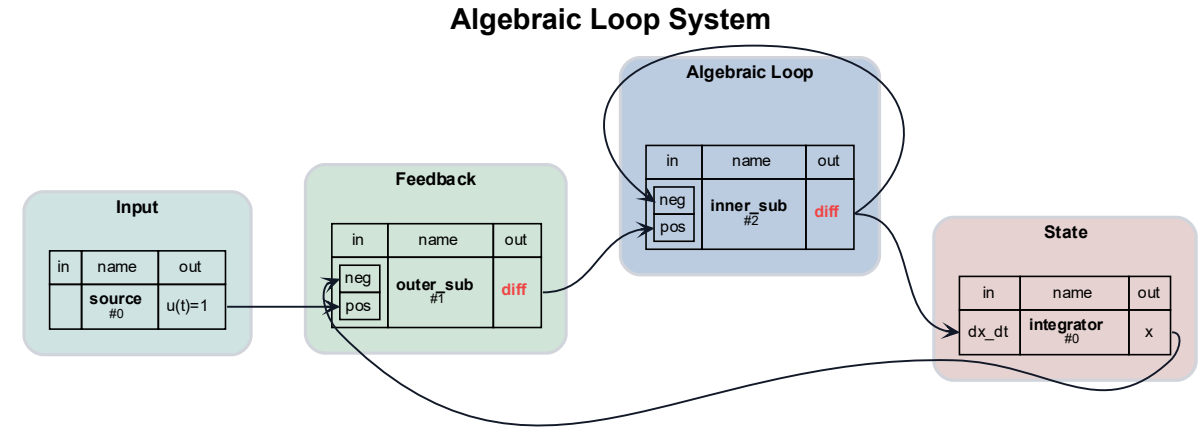
- The **Interface Jacobian-based Co-Simulation Algorithm (IJCSA)** generalizes the explicit schemes by applying a **Newton method at the interface level**, using the Jacobian of the interface constraints.
- Input and output relations of a co-simulation scenario with two simulation units

$$Y_1 = S_1(U_1), \quad Y_2 = S_2(U_2)$$
 - SUs $S_i(U_i)$ have internal state variables X_i and output variables Y_i
- Interface compatibility constraint equations

$$I_1(S_1(U_1), S_2(U_2), U_1, U_2) = 0, \quad I_2(S_1(U_1), S_2(U_2), U_1, U_2) = 0$$
 - The **simplest interface constraint** for two coupled SUs is described by (input of one SU must equal the output of the other SU)

$$I_1(S_2(U_2), U_1) = U_1 - Y_2 = 0, \quad I_2(S_1(U_1), U_2) = U_2 - Y_1 = 0$$
 - The interface **residual** is defined as

$$R_1 = I_1(S_1(U_1), S_2(U_2), U_1, U_2), \quad R_2 = I_2(S_1(U_1), S_2(U_2), U_1, U_2)$$
- As a result of **solving** the interface constraint equations, we **determine the set of input values U for each simulation unit**.



Iteration Sequence for the Newton Method

- ϕ : vector of unknowns
- r : residual vector
- J : Jacobian operator
- m : iteration index

$$\begin{aligned}
 {}^{m+1}\phi &= {}^m\phi - J(r({}^m\phi))^{-1}r({}^m\phi) \\
 J(r({}^m\phi))({}^{m+1}\phi - {}^m\phi) &= -r({}^m\phi) \\
 J(r({}^m\phi))\Delta^{m+1}\phi &= -r({}^m\phi) \\
 \phi &= \begin{bmatrix} U_1 \\ U_2 \end{bmatrix}, \quad r = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix}
 \end{aligned}$$

Linear Interface System for the Newton Method:

$$\begin{aligned}
 J(r({}^m\phi)) &= \begin{bmatrix} \frac{\partial R_1}{\partial U_1} & \frac{\partial R_1}{\partial U_2} \\ \frac{\partial R_2}{\partial U_1} & \frac{\partial R_2}{\partial U_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial I_1}{\partial U_1} & \frac{\partial I_1}{\partial U_2} \\ \frac{\partial I_2}{\partial U_1} & \frac{\partial I_2}{\partial U_2} \end{bmatrix} = \begin{bmatrix} \frac{\partial(U_1 - Y_2)}{\partial U_1} & \frac{\partial(U_1 - Y_2)}{\partial U_2} \\ \frac{\partial(U_2 - Y_1)}{\partial U_1} & \frac{\partial(U_2 - Y_1)}{\partial U_2} \end{bmatrix} = \begin{bmatrix} I & -\frac{\partial(Y_2)}{\partial U_2} \\ -\frac{\partial(Y_1)}{\partial U_1} & I \end{bmatrix} \\
 \begin{bmatrix} I & -\frac{\partial(Y_2)}{\partial U_2} \\ -\frac{\partial(Y_1)}{\partial U_1} & I \end{bmatrix} \begin{bmatrix} \Delta U_1 \\ \Delta U_2 \end{bmatrix} &= - \begin{bmatrix} \Delta R_1 \\ \Delta R_2 \end{bmatrix}
 \end{aligned}$$

Algorithm 2.1: Interface Jacobian-based Co-Simulation Algorithm for a two-code example.

```

// Time loop
1 for n = 0 to n = n_end do
    // Iteration loop
    2 for m = 0 to m = m_end do
        // Solve for all subsystems in parallel
        3  ${}^mY_1^{n+1} = S_1({}^mU_1^n)$ 
        4  ${}^mY_2^{n+1} = S_2({}^mU_2^n)$ 
        // Compute and check residual
        5  ${}^m r^n = \begin{bmatrix} {}^m R_1^n \\ {}^m R_2^n \end{bmatrix} = \begin{bmatrix} {}^mU_1^n - {}^mY_2^{n+1} \\ {}^mU_2^n - {}^mY_1^{n+1} \end{bmatrix}$ 
        6 if  $\|{}^m r^n\| < \epsilon$  then
            7 break
        // Get part of interface Jacobian from each subsystem
        8  ${}^m J_1^n = \mathcal{J}(S_1({}^mU_1^n)) = \begin{bmatrix} \frac{\partial Y_1}{\partial U_1} & \frac{\partial Y_1}{\partial U_2} \end{bmatrix}$ 
        9  ${}^m J_2^n = \mathcal{J}(S_2({}^mU_2^n)) = \begin{bmatrix} \frac{\partial Y_2}{\partial U_1} & \frac{\partial Y_2}{\partial U_2} \end{bmatrix}$ 
        // Assemble global interface Jacobian
        10  ${}^m J_{\text{global}}^n = \mathcal{A}_J({}^m J_1^n, {}^m J_2^n) = \begin{bmatrix} I & -\frac{\partial Y_2}{\partial U_2} \\ -\frac{\partial Y_1}{\partial U_1} & I \end{bmatrix}$ 
        // Solve for corrector
        11  ${}^m \Delta c^n = -({}^m J_{\text{global}}^n)^{-1} {}^m r^n$ 
        // Apply corrector
        12  ${}^{m+1}U_1^n = {}^mU_1^n + {}^m \Delta c_1^n$ 
        13  ${}^{m+1}U_2^n = {}^mU_2^n + {}^m \Delta c_2^n$ 
        // Initial solution for next time step
        14  ${}^0U_1^{n+1} = {}^{m_{\text{end}}+1}U_1^n$ 
        15  ${}^0U_2^{n+1} = {}^{m_{\text{end}}+1}U_2^n$ 
    
```

Main Features of Local IJCSA

- **SCC-Level Coupling Resolution:**
 - Solves algebraic loops **within individual Strongly Connected Components (SCCs)**
 - Each SCC represents a minimal algebraic subsystem with cyclic dependencies
 - Applied **before** each SCC's time integration step
- **Localized Newton Iteration:**
 - **Phase 1 (Interface-Newton):** Solve for consistent interface values within the SCC at time t with frozen states ($dt=0$)
 - **Phase 2 (Time Integration):** SCC components advance from t to $t + dt$ using converged values
 - Iteration scope limited to interface inputs within one SCC
- **Integration with Generation-Based Execution:**
 - Respects causality and execution order defined by generation levels
 - Solves algebraic loops on-demand when processing each generation
 - Components outside the SCC can proceed independently
- **Efficient and Modular:**
 - Small Jacobian matrices (only SCC interface inputs)
 - Lower computational cost per iteration
 - Independent SCC solutions can potentially be parallelized

Mathematical Formulation

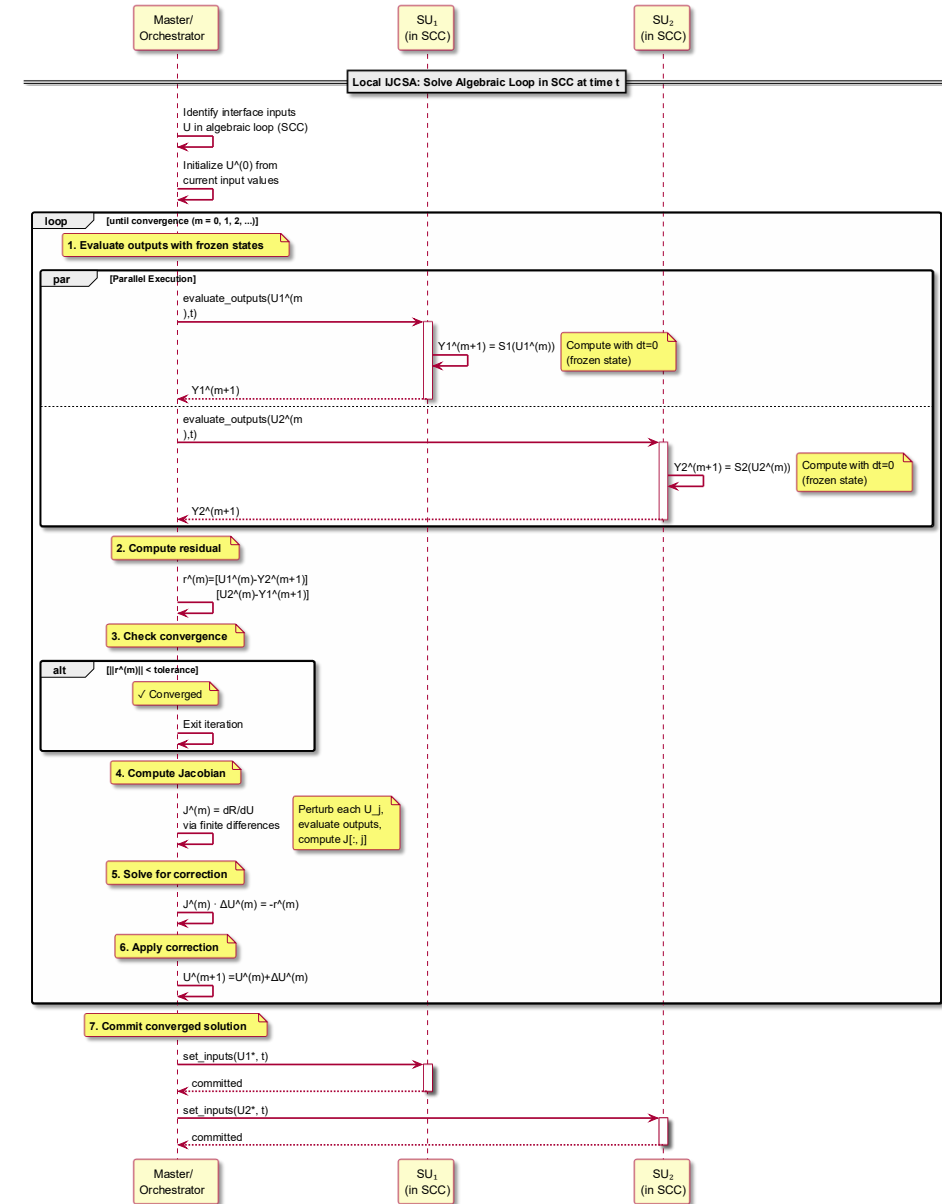
- For an SCC with components $\{S_1, S_2, S_3, \dots, S_k\}$ find interface inputs U_{SCC} such that:

$$r(U_{SCC}) = U_{SCC} - Y_{SCC}(U_{SCC}) = 0$$

- U_{SCC} = interface inputs on zero-delay connections within the SCC
- $Y_{SCC}(U_{SCC})$ = corresponding driver outputs evaluated with frozen states

Key Properties

- Scope: single SCC (minimal algebraic loop)
- Unknowns are the interface inputs (dst_comp , dst_port) on zero-delay connections within the SCC
- Jacobian approximation via finite differences and perturbation of $eps = 10^{-6}$
- Component's internal states are frozen ($dt=0$) during the iteration since the used FMUs exported from Modelica do not support state rollbacks

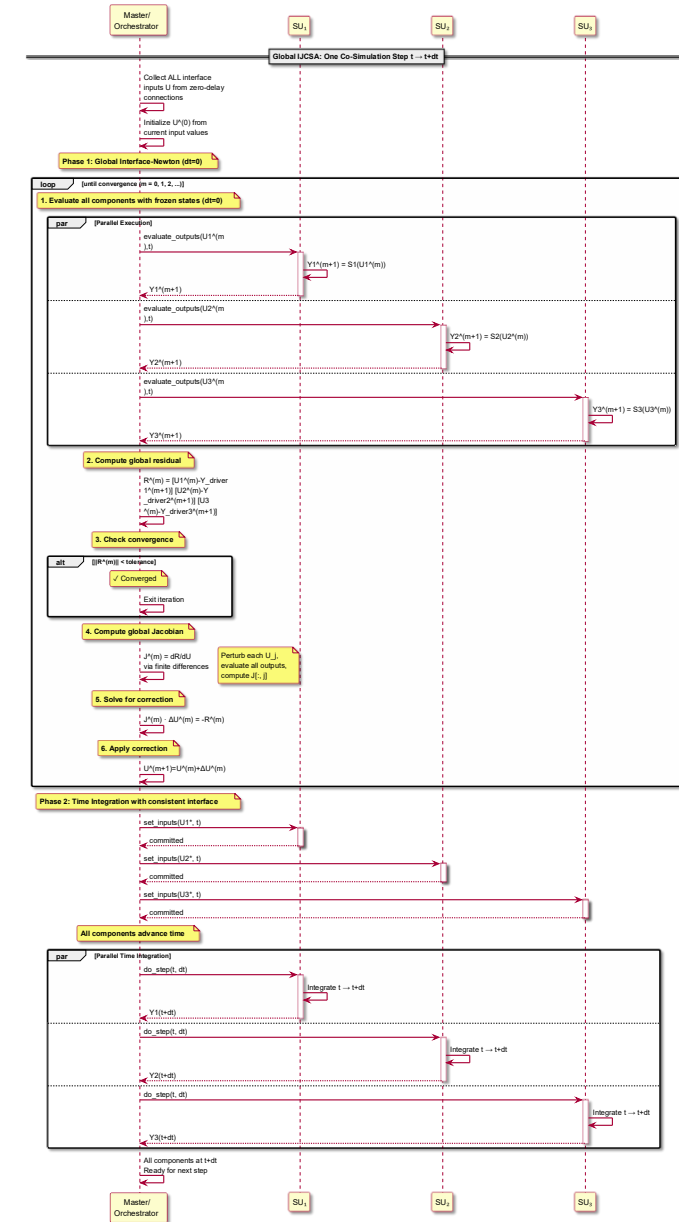


(Sicklinger, 2014))

Main Features of Global IJCSA

- **Two-Phase Structure:**
 - **Phase 1 (Interface-Newton):** Solve for consistent interface values at time t with frozen states ($dt=0$)
 - **Phase 2 (Time Integration):** All components advance from $t \rightarrow t + dt$ using the converged interface values
- **Global Scope:**
 - Considers **ALL** zero-delay interface connections in the system simultaneously
 - Not limited to individual strongly-connected components (SCCs)
 - Single Newton iteration covers the entire coupled system
- **Implicit Interface Coupling:**
 - Resolves algebraic constraints across the entire system
 - Ensures consistency of all interface values before time advancement
 - Handles complex coupling topologies with multiple feedback loops
- **Parallel Evaluation:**
 - All subsystems evaluated simultaneously during each Newton iteration
 - All subsystems integrate in parallel during Phase 2

Aspect	Local IJCSA	Global IJCSA
Scope	Single SCC (algebraic loop)	Entire system (all zero-delay connections)
When Applied	Within generation, before do_step	Once per macro step, before all do_step calls
Interface Variables	Only unknowns within one SCC	All zero-delay interface inputs in system
Residual	$r = U_{SCC} - Y_{SCC}(U_{SCC})$	$R = U_{global} - Y_{global}(U_{global})$
Jacobian Size	Small (number of SCC interface inputs)	Large (all zero-delay connections)
Computational Cost	Low (localized)	High (global system evaluation)
Integration	Components integrate after local solve	All components integrate after global solve
Use Case	Strongly-coupled subsystems with direct feedthrough	Entire system treated as one implicit step
Execution Order	Respects generation-based ordering	Single unified step, no generation ordering



(Sicklinger, 2014))

- An algebraic loop occurs when a signal loop in a models contains only blocks that have **direct feedthrough**.
- Direct feedthrough** refers to components that use the input value for the current time step to calculate the output value of the current time step.
 - If a signal loop contains only components with direct feedthrough, **circular dependencies** of output and input values in the same time step arise.
 - The resulting equation is an algebraic equation, that requires a solution at each time step and adds computational costs.
- Examples:** Math functions, Gains, Adders, **Subtractors**, etc.
- Non-examples:** Integrators, Delays, etc. (these do not have direct feedthrough)

Simple Algebraic Loop Example

- The system consists of
 - a source that outputs a constant value of 1,
 - an inner and outer subtractor unit, and
 - an integrator connected in a feedback loop

$$\begin{aligned} u(t) &= 1 \\ \dot{x}(t) &= x_a \\ 0 &= u(t) - x(t) - 2x_a(t) \end{aligned}$$

From the algebraic equation, we obtain:

$$x_a(t) = \frac{u(t) - x(t)}{2} = \frac{1 - x(t)}{2}$$

Substituting this into $\dot{x}(t) = x_a(t)$:

$$\dot{x}(t) + \frac{1}{2}x(t) = \frac{1}{2}$$

- With $u(t) = 1$ and $x(0) = 0$ we can solve this ODE.
- The integrating factor is $e^{\frac{1}{2}t}$. Multiplying both sides of the ODE by the integrating factor:

$$\frac{d}{dt} \left(e^{\frac{1}{2}t} x(t) \right) = \frac{1}{2} e^{\frac{1}{2}t}$$
- Integrating both sides with respect to t :

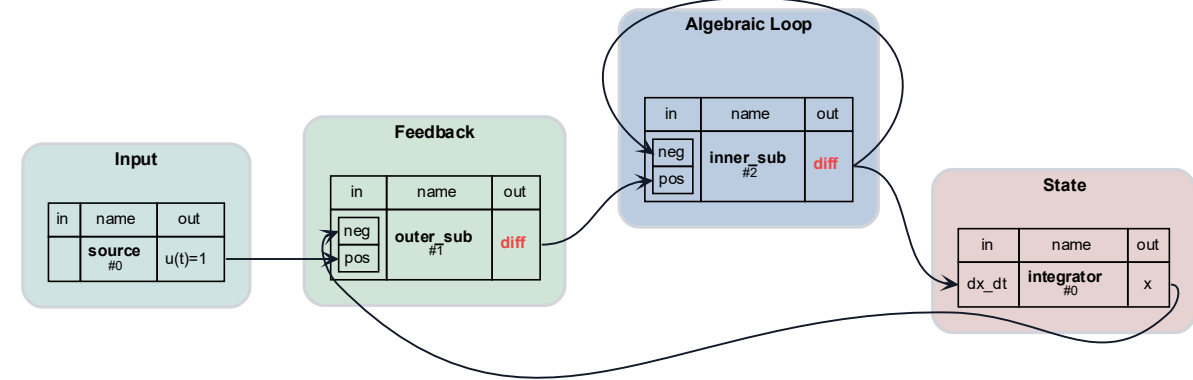
$$e^{\frac{1}{2}t} x(t) = e^{\frac{1}{2}t} + C$$
- Solving for $x(t)$:

$$x(t) = 1 + C e^{-\frac{1}{2}t}$$
- Applying the initial condition $x(0) = 0$:

$$0 = 1 + C \implies C = -1$$
- We obtain the analytical solution:

$$x(t) = 1 - e^{-\frac{1}{2}t}$$

Algebraic Loop System



```
=====
Solving algebraic SCC ['inner_sub']:

Connections:
Interface inputs: [('inner_sub', 'neg')]
Internal zero-delay connections: [('inner_sub', 'diff', 'inner_sub', 'neg')]
External zero-delay inputs: [('outer_sub', 'diff', 'inner_sub', 'pos')]

Initial guess for inner_sub.neg = 0.0

Starting IJCSA iterations:
Iteration 0: u = [0.]
Residual for inner_sub.neg: -1.0
Residual for inner_sub.neg: -0.9999979999999999
Jacobian J = [[2.]]
Correction Δu = [0.5]
Iteration 1: u = [0.5]
Residual for inner_sub.neg: -2.8755664516211255e-11
Converged in 1 iterations.

Committed solved input inner_sub.neg = 0.49999999998562217
=====
Initial consistent system state:
Source;           {'u(t)=1': 1}
Outer Subtractor; {'diff': 1.0}
Inner Subtractor; {'diff': 0.5000000000143778}
Integrator;       {'x': 0.0}
```

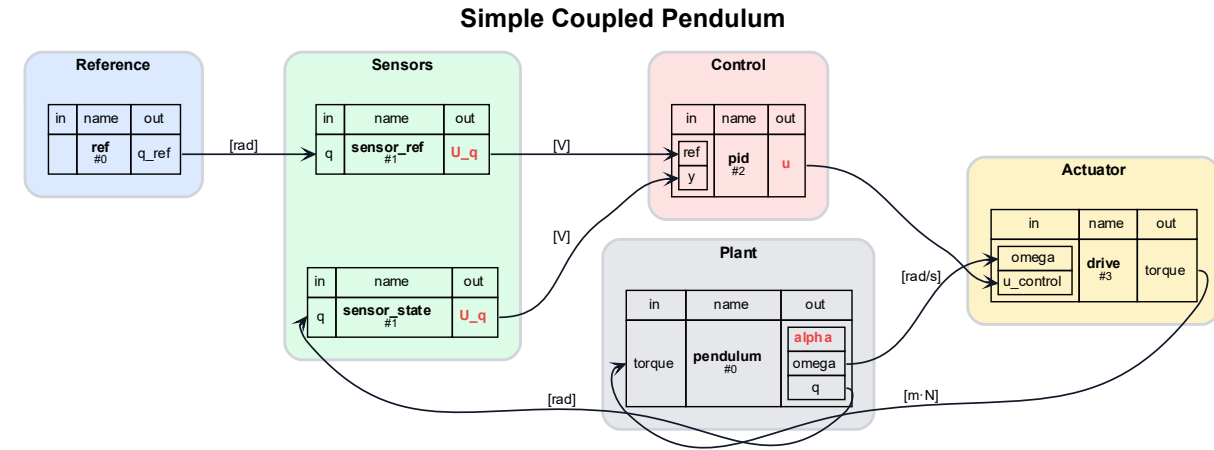
(MathWorks, 2025)

Pendulum Dynamics:

$$\frac{d\theta_p}{dt} = \omega_p, \quad \frac{d\omega_p}{dt} = -\frac{m \cdot g \cdot L}{I} \cdot \sin(\theta_p) + \frac{T_d}{I}$$

Simple Coupling between Pendulum and Drive

- DC motor with gearbox acting as torque element on the pendulum
 - Drive output torque T_{drive} is the input of the pendulum subsystem
 - Angular velocity ω is an input of the drive subsystem
- 1. **Voltage control:** $U = U_M \cdot u_{control}$
 - U : actual motor voltage
 - U_M : motor voltage
 - $u_{control} \in [-1, 1]$: normalized control signal
- 2. **Gearbox:** $\omega_M = i_M \cdot \omega_p$
 - motor angular velocity ω_M related to the pendulum ω_p through the gear ratio i_M
- 3. **Electromotive Force (back voltage):** $E = k_e \cdot \omega_M$
 - Back Voltage E that opposes the applied voltage U depending on motor specific speed constant k_e and motor speed ω_M (**Law of Induction**)
- 4. **Current Dynamics:** $\frac{dI}{dt} = \frac{1}{L_M} \cdot (U - E - R_M \cdot I)$
 - $(U - E)$: Net voltage driving current (effective voltage available)
 - $R_M \cdot I$: Voltage drop across resistance (Ohm's Law)
 - $\frac{1}{L_M}$: Inductive time constant (small L_M allows fast current response)
 - Rest $E = 0$: full voltage $(U - R_M \cdot I)$ accelerates the current I
 - Steady speed $\omega_M = const.$: E balances applied voltage and current stabilizes
- 5. **Torque Generation:** $T_{drive} = \eta_M \cdot i_M \cdot k_M \cdot I$
 - Motor torque is proportional to armature current I , motor torque constant k_M , gear box ratio i_M , and motor efficiency η_M
 - k_M (Nm/A): motor torque constant, intrinsic motor property (depends on magnetic field strength)



Structural Dependencies

- **Direct feedthrough components:**
 - sensors for U_q , controller for $u_{control}$, and pendulum for α (not used as input)
 - Outputs at time t depend **instantaneously** on its input at time t (not only on states)
- The angular position θ_p and angular velocity ω_p depend only directly on their own internal states and **have no other input dependencies**.
- The derivative of the angular velocity $\frac{d\omega_p}{dt}$ depends on the input torque T_{drive} from the drive FMU.
- Since output variables of the pendulum subsystem do not have input dependencies on the drive, there is **no direct algebraic loop between the pendulum subsystem and the drive**
- **State dependencies** in the drive and pendulum **break direct feedthrough** and do not create an algebraic loop
- **Feedback of the angular position** via the sensor and PID controller is **no algebraic loop**, no circular dependencies within the same time step

(Blochwitz, 2012))

Pendulum Dynamics:

$$\frac{d\theta_p}{dt} = \omega_p, \quad \frac{d\omega_p}{dt} = -\frac{m \cdot g \cdot L}{I} \cdot \sin(\theta_p) + \frac{T_d}{I}$$

Complex Coupling between Pendulum and Drive

- The drive and pendulum subsystems are coupled via a shared coordinate

$$\theta_{drive} = \theta_{pendulum}, \quad \omega_{drive} = \omega_{pendulum}, \quad \alpha_{drive} = \alpha_{pendulum}$$

- Electrical Subsystem:** Same as in the simple coupled co-simulation.

- Mechanical Subsystem:**

- Motor inertia J_1 and interface inertia J_2 are considered
- Torsional spring-damper element connects motor and interface inertias.
- A prescribed-motion element (**Modelica.Mechanics.Rotational.Sources.Move**)

- Coupling Inputs:**

- Drive receives angular position θ , angular velocity ω , and angular acceleration α from the pendulum subsystem (describe the full kinematic state of the pendulum)

- Coupling Outputs:**

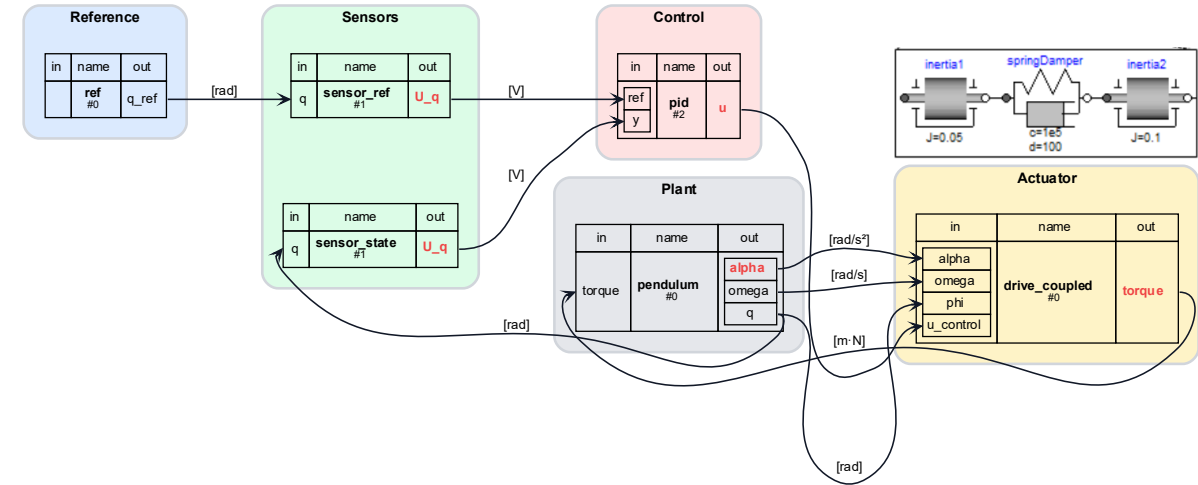
- Drive return the reaction torque T_{drive} to the pendulum subsystem.
- The torque is then fed back into the pendulum dynamics

- Drive Train of the Drive Subsystems:**

$T_M \rightarrow J_1 \rightarrow \text{Torsional Spring-Damper}(c, d) \rightarrow J_2 \rightarrow \text{Torque Sensor} \rightarrow \text{Prescribed-Motion Element}$

- The prescribed-motion is fed with the angular position, velocity, and acceleration from the pendulum subsystem θ, ω, α .
- This prescribed-motion element is **forced to follow exactly the pendulum motion**, creating a **rigid constraint** between the drive and pendulum subsystems.
- Mechanically identical to **connecting two rotational bodies with a rigid, massless link**.

Complex Coupled Pendulum System



Algebraic Coupling

- The **Move** receives the angular acceleration α_p as input from the pendulum and forces the interface to follow the same angular acceleration.
- The torque depends instantaneously on, spring-damper forces, which depend on the relative angular position and velocity between motor and interface, and inertial torque from the interface inertia

$$T_{drive} = J_2 \cdot \alpha_p + c \cdot (\theta_{interface} - \theta_{motor}) + d \cdot (\omega_{interface} - \omega_{motor}) + J_1 \cdot \alpha_{motor}$$

$$T_{drive} = f(\alpha_p), \quad \alpha_p = g(T_{drive})$$

- We can write the **coupled system** as:

$$T_{drive} = J_2 \cdot \alpha_p + f_1(x_{drive}), \quad \alpha_p = f_2(x_p, T_{drive})$$

- This gives the **classical algebraic loop structure that must be solved simultaneously**:

$$T_{drive} = J_2 f_2(x_p, T_{drive}) + f_1(x_{drive})$$

- $f_1(x_{drive})$ represents all terms in the torque equation that depend only on the internal states and parameters of the drive subsystem.
- $f_2(x_p, T_{drive})$ represents all terms in the pendulum acceleration equation that depend on the internal states of the pendulum subsystem and the input torque T_{drive} .

(Blochwitz, 2012))

White-Box Integration:

- In Modelica, all subsystems are **white-box models**
- All equations (differential and algebraic) are solved **simultaneously** by the DASSL solver
- No artificial coupling delays or communication step limitations
- Direct access to Jacobian information for implicit solving

Simultaneous Solution of Algebraic Loops:

$$T_{drive} = f(\alpha_{pendulum}), \quad \alpha_{pendulum} = g(T_{drive})$$

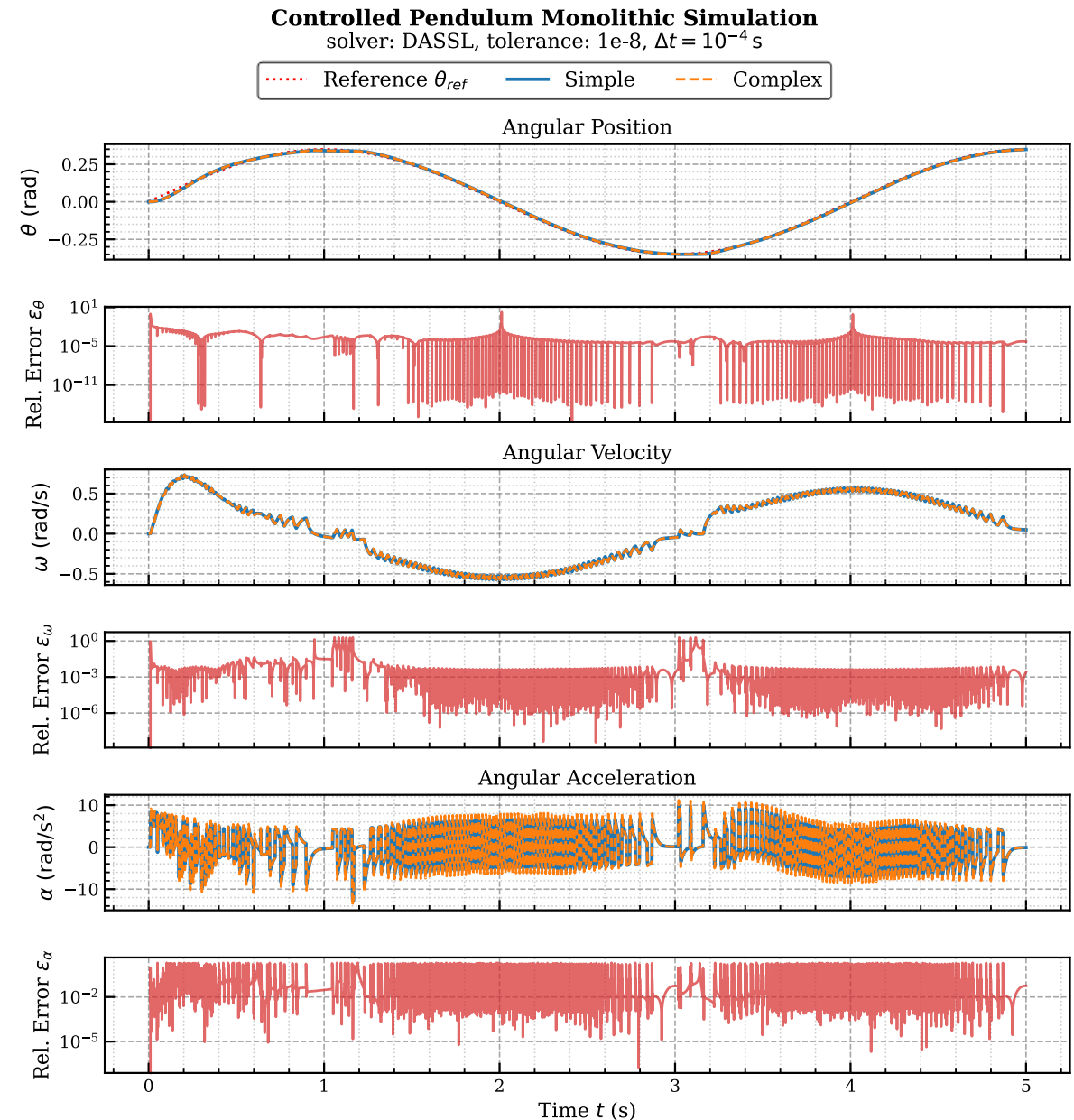
- DASSL solves this **implicitly at every integration step** using the full system Jacobian
- No interface iterations needed, algebraic constraints satisfied by construction

High Accuracy Integration:

- Solver:** DASSL (Differential-Algebraic System Solver)
 - Variable-order, variable-step BDF method (implicit)
 - Designed for stiff systems and DAEs
- Communication step:** $H = 1e-4$ s (output interval)
- Tolerance:** $1e-8$ (relative error control)
- Internal micro-steps:** Adaptive, typically much smaller than

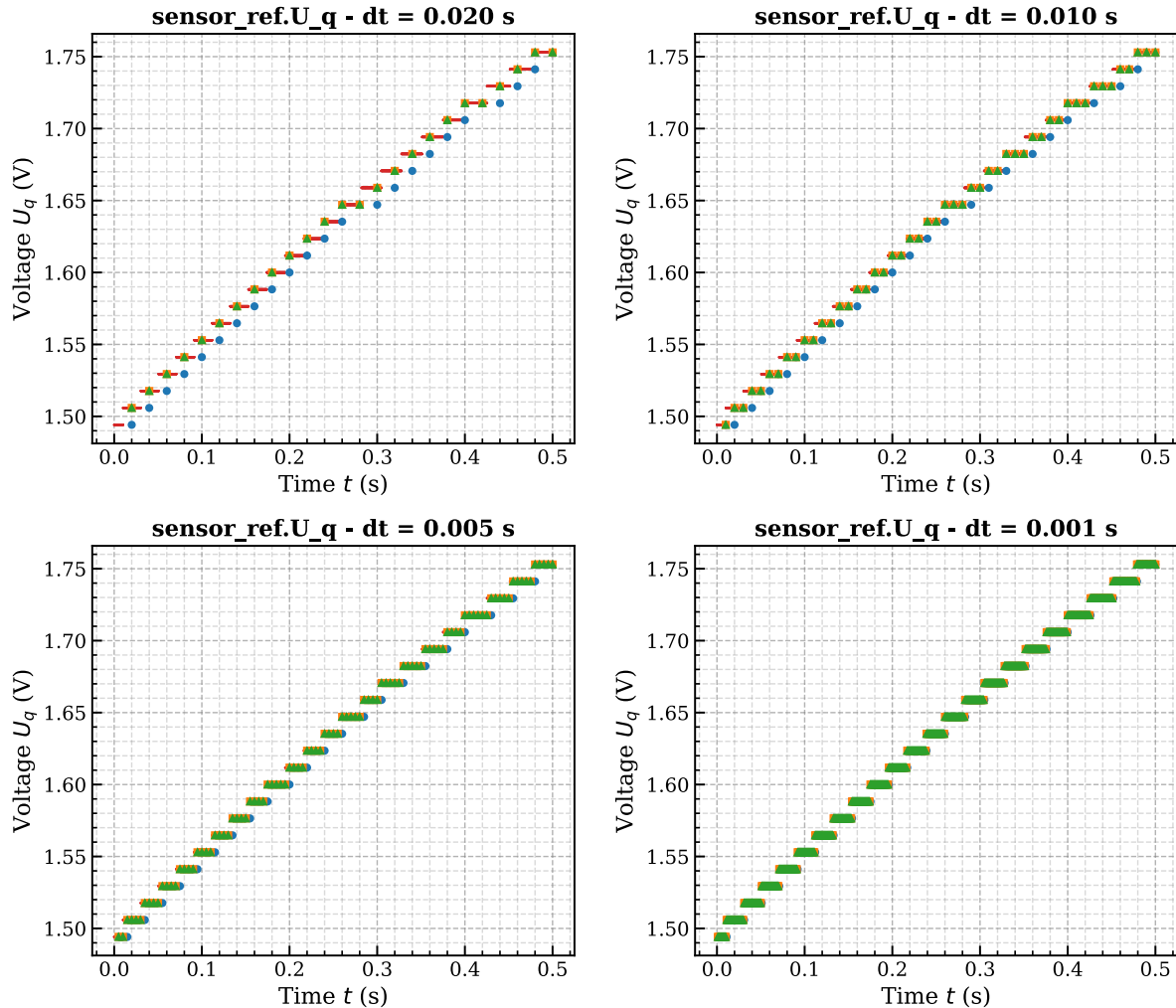
Verification Methodology

- Stage 1:** Verify monolithic models are consistent
 - Compare simple vs complex Modelica models
 - Both solved by DASSL with same tolerance
- Stage 2:** Verify co-simulation algorithms
 - Compare co-simulation results against monolithic reference
 - Convergence study: $H = \{0.02, 0.01, 0.005, 0.001\}$ s



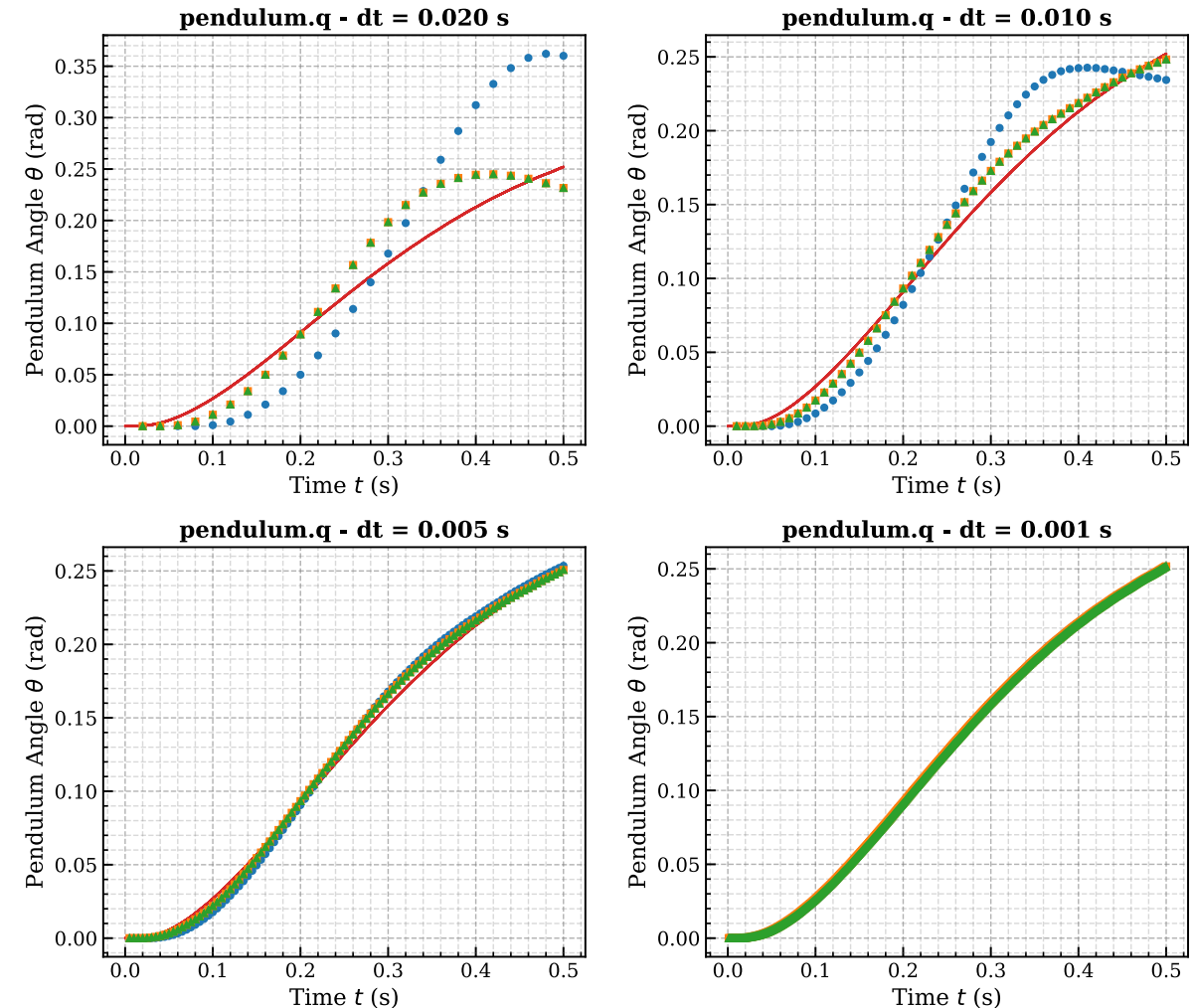
Trajectory Comparison for sensor_ref.U_q
Case: Complex

- Monolithic: DASSL (Modelica)
- Co-Simulation: Gauss-Seidel
- Co-Simulation: Jacobi
- Co-Simulation: IJCSA



Trajectory Comparison for pendulum.q
Case: Complex

- Monolithic: DASSL (Modelica)
- Co-Simulation: Gauss-Seidel
- Co-Simulation: Jacobi
- Co-Simulation: IJCSA



Overall Behavior:

- All co-simulation algorithms are convergent for the complex coupled pendulum
- The relative error in $q(t)$ decreases monotonically as the communication step size Δt is reduced
- For $\Delta t = 10^{-3}s$ all trajectories are visually indistinguishable from the monolithic reference

Jacobi:

- Algorithm does not handle direct-feed throughs within the system
- There is a noticeable phase lag in the reference q_{ref} and pendulum angle q for large dt

Gauss-Seidel:

- Subsystems are advanced sequentially, downstream subsystems see already updated outputs within the same macro step
- Phase large compared to the Jacobi Algorithm is reduced; trajectories are visibly closer to the reference

IJCSA:

- Same behavior as the Gauss-Seidel algorithm since GS already considers direct feed-through and algebraic loops through the local IJCSA implementation

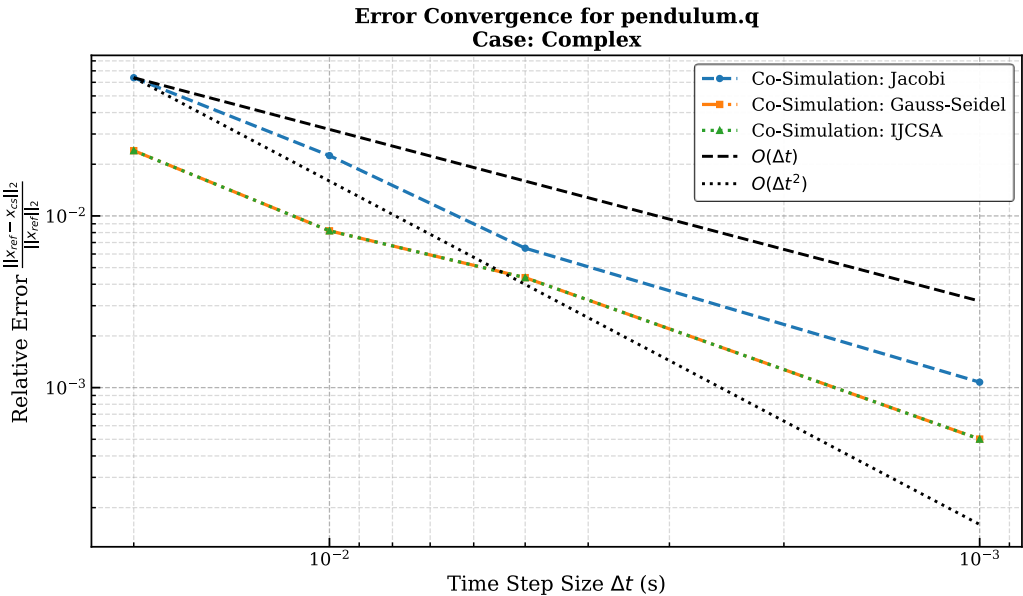
Convergence Behavior

■ Gauss-Seidel and IJCSA:

- Error curves are almost parallel to the $O(dt)$ reference line over the tested range
- First-order convergence w.r.t. the communication step size

■ Jacobi:

- For larger steps, the slope is close to $O(dt)$, but the absolute error is higher than for GS/IJCSA because of the extra macro-step delay
- For intermediate steps, the slope is between $O(dt)$ and $O(dt^2)$, indicating that
 - the splitting error is shrinking
 - the remaining error has a stronger contribution from the internal FMU solvers and the reference solution
- For smallest step size, Jacobi converges but stays above the GS/IJCSA curves



Aspect	Monolithic (DASSL)	Co-Simulation
Component Access	White-box (full equations)	Black-box (FMU interface only)
Algebraic Loops	Solved implicitly with full system Jacobian	Requires interface iterations (IJCSA)
Time Integration	Single global integrator with adaptive micro-steps	Multiple local integrators (one per FMU) with fixed macro step H
Jacobian Information	Full dense system Jacobian (all equations)	Local interface Jacobians only ($\partial Y/\partial U$)
Coupling Error	None (simultaneous solution within tolerance)	Depends on H and algorithm (explicit delay or iteration convergence)
Stability	A-stable (BDF methods)	Depends on coupling algorithm and macro step size
Step Size	Adaptive (typically 10^{-6} to 10^{-4} s internally)	Fixed macro step H (0.001 to 0.02 s in our study)
Equation System	Solves full DAE system simultaneously	Solves subsystems sequentially or iteratively

Framework Core Expansion:

- **New Components:**
 - OpenSimComponent: General Component Class for OpenSim models ✓
 - FEMComponent: General Component Class for netgen/NGSolve FEM models
- **Multicomponent Concept:**
 - Combine several representations of the same “thing” (e.g., pendulum FMU + OpenSim + FEM)
 - Implement state synchronization and rule-based switching between models ✓
- **Hybrid Capabilities:** 
 - Support discrete and continuous states
 - Introduce hybrid DAE event handling (zero-crossing detection, event-triggered switching)
- **File-based (FEM) simulation tool integration concept** 
 - Develop a workflow for file-based (FEM) model integration (openCFS, CalculiX, etc.)

Simulation Engine & Computational Causality:

- **Goal: Enable robust, adaptive, and event-driven co-simulation**
- **Multi-rate / Adaptive Time Stepping:**
 - Each component advances with its local solver time base ✓
 - synchronized at communication points
- **Causality Solver:**
 - Extend execution order computation to handle cyclic dependencies ✓
 - Algebraic loop resolution

User Interaction & System Assembly

- **Goal: Provide an intuitive, guided workflow inside Jupyter**
- **Multi-Step Wizard Interface (Standard & Expert Mode)**
 - Model selection -> FMU Export -> system assembly -> configuration -> simulation
 - Live preview of Modelica code 
- **Configuration Management:**
 - Save and reload full system configurations (components, connections, parameters)
 - Implement configuration validation
- **Visualization:** ✓
 - Generate system graphs
 - Enable live plotting and simulation diagnostics for monitoring
- **Result Handling:** 
 - Standardize simulation metadata and result exports for traceability

Framework Integration & Validation

- **Goal: Consolidate core modules and ensure framework stability** ✓
- **Testing and Automation:**
 - Update and expand unit tests for all framework layers
 - Unit tests for core classes, wrappers, and port connections
 - Integration tests for system assembly and simulated run
 - Cross-platform validation (Windows/Linux/Mac)
- **Data Layer:** Structured result logging for reproducible simulation data
- **Documentation and Packaging:** 
 - Refactor into modular pyproject-based package
 - Add tutorials, notebook, and licensing for distribution

1. Sicklinger, S., Belsky, V., Engelmann, B., Elmqvist, H., Olsson, H., Wüchner, R. and Bletzinger, K.-U. (2014), **Interface Jacobian-based Co-Simulation**. Int. J. Numer. Meth. Engng, 98: 418-444. <https://doi.org/10.1002/nme.4637>
2. Cláudio Gomes, Casper Thule, David Broman, Peter Gorm Larsen, and Hans Vangheluwe. 2018. **Co-Simulation: A Survey**. ACM Comput. Surv. 51, 3, Article 49 (May 2019), 33 pages. <https://doi.org/10.1145/3179993>
3. D. Broman *et al.*, "**Determinate composition of FMUs for co-simulation**," *2013 Proceedings of the International Conference on Embedded Software (EMSOFT)*, Montreal, QC, Canada, 2013, pp. 1-12, doi: 10.1109/EMSOFT.2013.6658580.
4. Blochwitz, T. & Otter, Martin & Åkesson, Johan & Arnold, Martin & Clauss, Christoph & Elmqvist, Hilding & Friedrich, Markus & Junghanns, Andreas & Mauss, Jakob & Neumerkel, Dietmar & Olsson, Hans & Viel, Antoine. (2012). **Functional Mockup Interface 2.0: The Standard for Tool independent Exchange of Simulation Models**. Proceedings. 10.3384/ecp12076173.
5. MathWorks. (2025). **Algebraic loops (Simulink)**. Simulink documentation. <https://de.mathworks.com/help/simulink/ug/algebraic-loops.html>
6. OpenModelica Association. (n.d.). **Solving Modelica models – DASSL** (OpenModelica User's Guide). OpenModelica documentation. <https://openmodelica.org/doc/OpenModelicaUsersGuide/latest/solving.html#dassl>